



南京大學

本科畢業設計

學院 智能軟件與工程學院

專業 軟件工程（智能化軟件）

題目 基於 k-kick 哈希結構的冷熱分
層存儲設計與數據庫集成實驗

年級 2022 學 號 221900323

學生姓名 紀澤操

指導教師 劉明謀 職 稱 副教授

提交日期 2026 年 6 月 3 日



南京大学本科毕业论文（设计） 诚信承诺书

本人郑重承诺：所呈交的毕业论文（设计）（题目：基于 k-kick 哈希结构的冷热分层存储设计与数据库集成实验）是在指导教师的指导下严格按照学校和学院有关规定由本人独立完成的。本毕业论文（设计）中引用他人观点及参考资源的内容均已标注引用，如出现侵犯他人知识产权的行为，由本人承担相应法律责任。本人承诺不存在抄袭、伪造、篡改、代写、买卖毕业论文（设计）等违纪行为。

作者签名：纪泽操

学号：221900323

日期：2026 年 6 月 3 日

南京大学本科生毕业论文（设计、作品）中文摘要

题目：基于 k-kick 哈希结构的冷热分层存储设计与数据库集成实验

学院：智能软件与工程学院

专业：软件工程（智能化软件）

本科生姓名：纪泽操

指导教师（姓名、职称）：刘明谋 副教授

摘要：

k-kick 树相关理论表明，在插入和删除操作允许进行少量元素迁移的条件下，可以在保证常数时间查询的同时，获得更优的时间与空间权衡。本文在原有 k-kick 紧凑哈希表简化复现的基础上，进一步设计并实现了一种结合 hot tail cubby 与 SQLite 的冷热分层存储方案。该方案保留 Facility 分区、tail cubby 和局部路由等核心思想，将 tail cubby 改造为内存热缓存，用于存储近期插入的数据以及具有访问局部性的数据；同时将冷数据下沉至 SQLite 数据库中，以获得持久化能力。为验证该设计的实际效果，本文实现了三种对照方案，分别为原始多层级 cubby 哈希结构、纯 SQLite 存储结构，以及 hot tail cubby + SQLite 混合结构，并在相同的工作负载和评价指标下，从插入延迟、查询延迟、删除延迟和平均每键空间占用等方面进行实验比较。实验结果表明，hot tail cubby + SQLite 混合方案在保留数据库持久化能力的同时，能够利用内存热缓存降低热数据的访问成本，各项测试指标均优于纯 SQLite 方案。与原始多层级 cubby 结构相比，该方案的平均每键空间占用增加，但插入和删除延迟有所降低。

关键词：k-kick 树;tail cubby; 冷热分层存储; 数据库集成

南京大学本科生毕业论文（设计、作品）英文摘要

THESIS: Design of Hot-Cold Tiered Storage Based on k-kick Hashing and Database Integration Experiments

DEPARTMENT: School of Intelligent Software and Engineering

SPECIALIZATION: Software Engineering (Intelligent Software)

UNDERGRADUATE: Ji Zecao

MENTOR: Liu Mingmou, Associate Professor

ABSTRACT:

The theory of k-kick tree shows that, when a small amount of element migration is allowed during insertion and deletion, a better trade-off between time and space can be achieved while preserving constant-time query performance. Based on a simplified reproduction of the original k-kick compact hash table, this thesis further designs and implements a hot-cold hierarchical storage scheme that combines a hot tail cubby with SQLite. The proposed scheme preserves the core ideas of Facility partitioning, tail cubby storage, and local routing. Specifically, the tail cubby is transformed into an in-memory hot cache, which is used to store recently inserted data and data with access locality. Meanwhile, cold data is moved into an SQLite database to provide persistence. To evaluate the effectiveness of the proposed design, this thesis implements three comparative schemes: the original multi-tier cubby hash structure, a pure SQLite storage structure, and the hot tail cubby + SQLite hybrid structure. Under the same workloads and evaluation metrics, these schemes are compared in terms of insertion latency, query latency, deletion latency, and average space usage per key. The experimental results show that the proposed hot tail cubby + SQLite hybrid scheme preserves the persistence capability of the database while reducing the access cost of hot data through the in-memory hot cache, and all tested metrics are better than those of the pure SQLite scheme. Compared with the original multi-tier cubby structure, the proposed scheme has higher average space usage per key, but it achieves lower insertion and deletion latencies.

KEYWORDS: k-kick tree; tail cubby; hot-cold tiered storage; database integration

目 录

目 录	III
第一章 引言	1
1.1 实验背景及意义	1
1.2 国内外现状	1
1.3 本文组织结构	2
第二章 问题模型与理论基础	4
2.1 探测复杂度与球槽分配模型	4
2.2 系统核心设定：全域与哈希基础	5
2.2.1 全局参数	5
2.2.2 可逆置换与全局路由哈希	5
2.2.3 Facility 编号与 Cubby 内局部地址	6
2.3 核心组件层级关系	6
2.4 商压缩的信息论思想	8
2.5 k-kick、MiniArray 与 Local Query Router	8
2.5.1 k-kick：逻辑分层与交换上界	8
2.5.2 MiniArray M	9
2.5.3 Local Query Router: $O(1)$ 查询与映射存储	9
2.5.4 辅助结构同步不变量	10
2.6 本章小结	10
第三章 哈希表结构与算法设计	11
3.1 Facility 与 MiniArray	11
3.1.1 Facility 的基本属性	11

3.1.2	MiniArray 的逻辑结构	11
3.1.3	Access: 按逻辑下标读取	12
3.1.4	Insert: 插入与叶分裂	12
3.1.5	Delete: 删除与叶合并	13
3.1.6	Update: 原地覆写	14
3.1.7	Split 与 Merge	14
3.1.8	Rank 与 Select	14
3.1.9	复杂度与空间性质	15
3.2	多层次 Cubby 设计	15
3.2.1	层级属性与初始状态	15
3.2.2	动态调整: 合并与拆分	16
3.2.3	Cubby 内部组件	16
3.2.4	插入与删除操作	17
3.3	k-kick 层级插入算法	17
3.3.1	层级 bin 定义	18
3.3.2	偏好序列	18
3.3.3	探测位置序列	19
3.3.4	bin 饱和与最大可用深度	19
3.3.5	随机深度选择与 InsertKick	20
3.3.6	ReInsert: 被踢元素向上放入	20
3.3.7	复杂度与正确性	22
3.4	Local Query Router	22
3.4.1	设计目标	22
3.4.2	映射规则	23
3.4.3	二叉前缀树存储与接口	23
3.4.4	复杂度与 Patricia 压缩	23
3.5	商压缩与键恢复	25
3.5.1	设计动机与信息分工	25
3.5.2	$\pi(x)$ 与 $g^*(x)$ 的位段划分	25
3.5.3	MiniArray B 中的 MetaEntry	26

3.5.4	恢复：重建 $\pi(x)$	27
3.5.5	查询校验	27
3.5.6	空间性质	28
3.6	插入、查询与删除	28
3.7	本章小结	29
第四章	系统性能实验	30
4.1	实验目的与环境	30
4.2	实验方案	30
4.2.1	实验分组与工作量	30
4.2.2	评价指标	31
4.3	规模 n 实验	31
4.4	Facility 规模 K 实验	32
4.5	k-kick 深度 k 实验	33
4.6	多层级 Cubby 动态重构实验	34
4.7	本章小结	35
第五章	总结与展望	36
5.1	工作总结	36
5.2	不足与局限	36
5.3	展望	37
	参考文献	38
	致 谢	42

第一章 引言

1.1 实验背景及意义

哈希表是一种通过哈希函数把关键字映射到数组相应位置的结构^[1-2]，它在数据库索引、键值存储、编译器符号表以及运行时系统等常见基础架构中具有广泛的应用场景。面向键值存储的哈希索引优化也是国内研究热点之一，相关工作从键摘要压缩与热点感知调度等角度改进空间利用与探查效率^[3]。相较于链地址法，开放寻址哈希表将元素直接放置在连续数组中，其查询路径相对较短，缓存局部性表现更优；但当负载因子上升时，冲突及探测长度会快速增加操作成本。工程上常采用预留空槽以降低冲突概率，但也会牺牲空间利用率。因此，哈希表设计需要做到：在查询、插入及删除时间可控的前提下，减少每个元素定位、更新与键恢复所需的额外存储。

Bender 等人的论文^[4]从理论层面阐述了上述问题：在保证查询时间复杂度为 $O(1)$ 的同时，若插入和删除允许承担 $O(k)$ 次元素交换代价（一次更新过程中被搬移的元素个数），则每个元素所占的额外空间可从 $O(\log \log n)$ 比特降至 $O(\log^{(k)} n)$ 比特（本文空间开销均按比特计； $\log^{(k)} n$ 表示对 n 连续取 k 次对数）。该结果表明，开放寻址哈希表的空间利用率不仅取决于空位占比，还与元素搬移以及恢复原始键信息的编码方式密切相关。此前，紧凑哈希表在常数时间操作下每键约需 $O(\log \log n)$ 比特冗余；动态检索与 filter 的下界^[5]、Iceberg 系列结果^[6-7] 以及紧凑检索结构^[8] 等已逐步逼近该信息论下界。本文在该理论框架下给出 Facility-Cubby-Slot 分层结构的简化实现，具体架构与模块分工见下一节，各模块的算法细节留待第二、三章展开。

1.2 国内外现状

哈希表实现主要包括链地址法与开放寻址法两大类^[1-2]。链地址法借助链表或桶结构处理冲突，实现直接，但指针与动态分配会带来额外空间和缓存开销。

开放寻址法将元素置于数组槽位，局部性较好，却易受负载因子、探针序列与删除策略影响。布谷鸟哈希^[9-10]以多候选位置与踢出过程缩短查询路径；最小完美哈希^[11-13]面向静态集合提供更强大查找保证，其动态扩展与紧凑编码也有持续研究^[5,14]；打包与压缩哈希表^[15]、IcebergHT^[6-7]等工程方案则侧重实际吞吐与空间折中。国内方面，GPU环境下的动态哈希表索引^[16]等工作从紧凑型数据结构与分页显存管理提升大规模并发访问的可扩展性。这些路线分别改进了查询确定性、构造效率或空间利用率，但在同时支持动态插入删除与降低每键空间冗余之间仍难以兼顾。

近年来，研究进一步聚焦到位级空间开销：仅靠负载因子无法刻画紧凑哈希表的真实成本，因为系统还须保存支持查询、更新与键恢复的辅助结构。Bender等人的最优时间/空间权衡框架^[4]将元素放置建模为带删除的球槽分配^[17-18]，并以k-kick、局部查询路由器（Local Query Router）、MiniArray与商压缩组合出可调的权衡曲线。本文工作定位于该框架的简化工程实现与实验观察：在Facility-Cubby-Slot分层组织下，分别实现MiniArray变长元数据维护、k-kick有界交换插入与Local Query Router常数时间查询，并以操作延迟、踢出次数等指标对照理论权衡曲线^[4-5]。

1.3 本文组织结构

本文以Bender等人的论文^[4]的关键构造为核心，给出可运行的简化原型并完成实验评估。实现沿用Facility-Cubby-Slot结构，实现MiniArray维护变长元数据、Cubby内k-kick完成有界交换插入、Local Query Router将查询降为 $O(1)$ 、槽位存放商压缩载荷、Cubby内部的MiniArray B 保存键恢复差分与扩缩容采用active/old双表渐进式迁移等。

本文针对三个相互联系的问题依次展开论述：

1. 论文各模块的理论模型如何落地，并合并成一个完整的系统。
2. 在MiniArray、k-kick与Local Query Router完整实现的前提下，插入、查询与删除各阶段的延迟与吞吐处于何种量级。
3. 在不同的 n 、 K 、 k 分组实验下，k-kick踢出次数、Cubby拆分与合并次数等指标变化趋势如何，是否满足 $O(k)$ 交换上界以及预期扩缩容行为。

后续章节安排如下：第二章介绍理论基础，第三章阐述哈希表结构与算法设计，第四章报告系统性能实验结果，第五章总结工作并讨论改进方向。

第二章 问题模型与理论基础

2.1 探测复杂度与球槽分配模型

开放寻址哈希表可被抽象为在线状态下的球槽分配问题^[17-19]：表中有固定数量的槽位，每个槽位至多容纳一个元素；系统需要支持动态插入与删除，且任一时刻元素个数不超过槽位数。对任意键 x ，哈希过程会为其生成一条预先确定的随机探测序列

$$h_1(x), h_2(x), \dots$$

元素只会被安置于该序列给出的候选槽位之一。

当某元素最终被置于其探测序列的第 i 个候选位置时，探测复杂度（查询时为记录“元素落在第 i 个候选位置”所需的比特数，记为 $\Theta(1 + \log i)$ ，与辅助元数据空间直接相关）定义为 $\Theta(1 + \log i)$ 。该量可理解为查询阶段为记录“元素采用了第几个候选位置”而必须保存的比特数目，因而与辅助元数据的空间开销直接相关。在该模型下，用平均探测复杂度刻画整体附加空间，用交换成本刻画单次更新中被搬移元素的个数；最优哈希表理论所关注的核心问题，是在允许有限交换的前提下，如何尽可能降低平均探测复杂度^[4-5,20]。一般而言，若表内 n 个元素分别落在其探测序列的第 $i_1, i_2, \dots, i_n$ 个候选位置，则平均探测复杂度为

$$\frac{1}{n} \sum_{i=1}^n \Theta(1 + \log i_i),$$

而单次插入或删除的交换成本为踢出链中被搬移元素个数的上界。Bender 等人^[4]通过调节 k-kick 深度参数 k ，使上述两项复杂度分别控制在 $O(\log^{(k)} n)$ 与 $O(k)$ 量级，形成可调的时间/空间权衡曲线。他们提出：若查询必须在常数时间内完成，且插入、删除允许承担 $O(k)$ 次元素交换，则每个元素所需的额外空间可由 $O(\log \log n)$ 比特降至 $O(\log^{(k)} n)$ 比特。该结果说明，开放寻址哈希表的空间效率不仅取决于空位占比，还与元素搬移、局部引导信息及键恢复编码密切相关。后

文将给出与之对应的地址拆分、分层存储与 Local Query Router 设计，使探测序列在插入阶段完整定义、同时满足查询阶段不必线性扫描。

2.2 系统核心设定：全域与哈希基础

2.2.1 全局参数

设键空间全域为 U ，且 $|U| = \text{poly}(n)$ 。当前表内元素规模提示为 n ；表容量 N 取为 2 的幂，使实际容量落在区间 $[N, 2N]$ 内。每个 Facility 的基准规模为

$$K = \text{polylog } n,$$

通常取 $K = \Theta(\log^c n)$ 且 $c \in [2, 4]$ ，以便在 Facility 数量 N/K 与局部结构规模之间取得平衡^[2,4]。

2.2.2 可逆置换与全局路由哈希

对任意键 x ，先经随机可逆置换函数 π 映射为 $\pi(x)$ 。该类置换可由 Feistel 等构造从伪随机函数生成^[21-23]。定义 $x[a, b]$ 表示 x 的第 a 位到第 b 位组成的子串（从低位到高位），则全局路由哈希 $g^*(x)$ 定义为 $\pi(x)$ 的低 $\log N$ 位：

$$g^*(x) = \pi(x)[1, \log N].$$

置换值在 $\log N$ 位以上的部分存入槽位载荷，记为

$$\text{storage} = \pi(x)[\log N + 1, \log U],$$

即商压缩意义下的 payload；该部分不再参与 Facility 与 Cubby 的路由，仅在命中校验时与局部元数据联立以恢复完整的 $\pi(x)$ 。

从比特布局上看， $\pi(x)$ 可示意划分为“高位 payload + 低位 $g^*(x)$ ”；而 $g^*(x)$ 自身又可自高向低继续划分为 Facility 编号段、Cubby 中间段与 Local Query Router 段，分别用于表级分区、Cubby 内桶化与 $A[i]$ 索引。图 2-1 概括了这一层次。

$\pi(x)$ 高位: payload / storage	$g^*(x) = \pi(x)[1, \log N]$	
$\pi(x)[\log N + 1, \log U]$	Facility 位	Cubby 中间位 + $g_I(x)$
存入槽位 storage	$r = g^*[\log K + 1, \log N]$	$g^*[1, \log K]$

图 2-1 $\pi(x)$ 与 $g^*(x)$ 的位段划分

2.2.3 Facility 编号与 Cubby 内局部地址

在 $g^*(x)$ 已确定的前提下，Facility 编号取

$$r = g^*(x)[\log K + 1, \log N] = \left\lfloor \frac{g^*(x)}{K} \right\rfloor.$$

对容量为 $|I|$ 的 Cubby，其内部 Local Query Router 段（与 k-kick 偏好序列无关）定义为

$$g_I(x) = g^*(x)[1, \log |I|],$$

即 $g^*(x)$ 的最低 $\log |I|$ 位，取值范围为 $[0, |I| - 1]$ 。所有满足 $g_I(x) = i$ 的键由该 Cubby 内第 i 号 Local Query Router $A[i]$ 管理； $g^*(x)$ 中 $[\log |I| + 1, \log K]$ 的中间位则与 Facility 编号 r 及 payload 共同用于恢复完整 $g^*(x)$ ，进而经由 π^{-1} 得到原始键 $x^{[4,8]}$ 。

2.3 核心组件层级关系

在逻辑结构上，整张表可视为一个大规模数组，均等划分为 (N/K) 个 Facility；每个 Facility 管理规模为 K 的局部范围，并在其内部维护多层级、不同容量的 Cubby 集合。Cubby 是实际存放元素的局部块；每个 Cubby 内部又包含以下部分：

- storage 数组：长度为 $|I|$ 的槽位数组，存放商压缩 payload 即 $\pi(x)[\log N + 1, \log U]$ ；
- k-kick 逻辑：将整个 cubby 逻辑上划分为多层级 bin，完成插入、踢出与被踢元素的重插，不要求额外的物理树结构；
- MiniArray A ：长度为 $|I|$ 的 Local Query Router 数组， $A[i]$ 维护所有 $g_I(x) = i$ 的键到探测序列下标的映射；
- MiniArray M ：记录各 k-kick 深度 bin 的空闲槽位信息，支持在常数时间内

判定 bin 内是否存在空位、并定位空位位置；

- MiniArray *B*: 与 storage 槽位一一对应, 保存 $\delta = g_I(x) - i$ 及恢复 $g^*(x)$ 所需的中间位等信息。

Facility 层另维护一棵基于 B 树、深度为 $O(1)$ 的 MiniArray, 用于在变长元数据与 Facility 级索引上实现均摊常数时间的访问、插入、删除与更新^[4,24]。MiniArray 中每个内部节点最多有多少个子节点称为 fanout, 取 $\Theta(\log n / \log \log n)$, 内部指针仅需 $\Theta(\log \log n)$ 比特, 从而到达整体 $O(\log^{(k)} n)$ 的空间目标^[25-26]。

为在动态扩缩容下维持高装填因子, 每个 Facility 还引入 *tail Cubby* 机制: 初始时仅含一个 1-tiered Cubby 作为 tail; 此后所有新插入默认进入 tail, tail 满后转为普通 1-tiered Cubby 并新建 tail。不同 *j*-tiered Cubby 的目标个数与容量分别为

$$t_j = \Theta\left(\frac{(\log^{(j+1)} n)^2}{(\log^{(j)} n)^2}\right), \quad r_j = \frac{K}{(\log^{(j)} n)^2};$$

当某层 Cubby 数量偏离 t_j 时, 通过向上合并或向下拆分并在新 Cubby 内重插全部元素来恢复分布不变量。删除非 tail 元素时, 从 tail 随机抽取一个元素回填至被删槽位; tail 为空时从 1-tiered 层提升一个 Cubby 作为新 tail, 并可能触发拆分。tier 调度、tail 不变量与 k-kick 踢出规则共同构成第三章的算法主体^[2,4,7,27]。

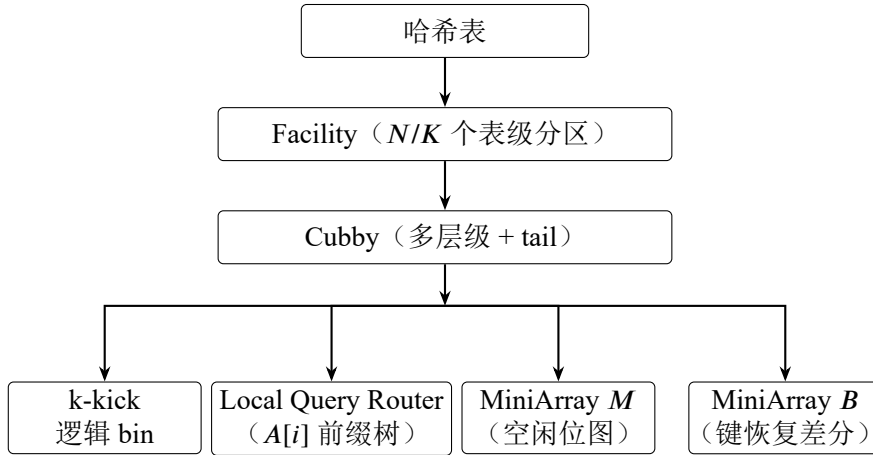


图 2-2 Facility-Cubby-Slot 分层架构

2.4 商压缩的信息论思想

如果在槽位中完整保存原始键，则部分信息与寻址过程重复。商压缩借助双射函数 π ，映射 x 为 $\pi(x)$ ，低 $\log N$ 位 $\pi(x)[1, \log N] = g^*(x)$ 的各位段交由地址上下文（Facility 编号、Cubby 容量、槽位下标）与局部元数据分担，槽位仅保存 payload 即 $\pi(x)[\log N + 1, \log U]$ 与无法由上下文恢复的差分信息。

具体地，当 $\text{storage}[i]$ 存放键 x 对应信息时，MiniArray $B[i]$ 保存

$$\delta = g_I(x) - i,$$

以及 $g^*(x)[\log |I| + 1, \log K]$ 的比特位。配合 Facility 编号 $r = g^*(x)[\log K + 1, \log N]$ ，即可重建

$$g^*(x)[1, \log N],$$

再加上 payload 得到 $\pi(x)$ ，经 π^{-1} 恢复 x 。该编码方式与紧凑检索结构^[8,11]、打包式哈希表^[13,15]所强调的“按条目实际长度计费”相同；但在 k-kick 动态搬移与 tier 重建下， B 、 A 与 M 必须同步更新，对 MiniArray 与 Local Query Router 的动态维护提出了比静态紧凑哈希更严格的要求^[4-5]。

2.5 k-kick、MiniArray 与 Local Query Router

2.5.1 k-kick：逻辑分层与交换上界

在 Cubby 内部，k-kick 并不引入物理上的多层数组，而是将 storage 上长度为 $|I|$ 的连续槽位按固定规则划分为 $k + 1$ 个逻辑深度：深度 0 的 bin（bin 指 Cubby 按不同深度划分的子区间）覆盖整个 Cubby，更深层 bin 由上一层均匀切分得到，规模 $s_i = \Theta((\log^{(i)} K)^6)$ 。对键 x （此处均指 $\pi(x)$ ）定义偏好序列

$$g_0(x) \rightarrow g_1(x) \rightarrow \cdots \rightarrow g_k(x),$$

其中 $g_0(x)$ 为整个 Cubby， $g_{i+1}(x)$ 为 $g_i(x)$ 经均匀切分后 x 所属的子 bin。探测位置序列 $h_1(x), h_2(x), \dots$ 按先深后浅顺序枚举各 bin 内槽位；若元素最终落在第 j 个候选位置，则其探测复杂度为 $\Theta(1 + \log j)$ 。

定义 bin 在已满且其中每个元素的 insert_depth 均不低于该 bin 对应深度时饱和 (IsSaturated); 贪心策略下 $\text{GetMaxUsableDepth}(x)$ 返回 x 可插入的、未饱和 bin 的最大深度。插入时, 系统先取随机哈希 $s(x) \in [0, k]$, 再令

$$\text{depth} = \min(\text{GETMAXUSABLEDEPTH}(x), s(x)),$$

在 bin $g_{\text{depth}}(x)$ 内写入; 若 bin 已满, 则踢出其中 insert_depth 最小的元素并调用 ReInsert 向上安置。Bender 等人^[4] 证明: 在参数满足其设定时, 整条踢出链至多 k 次, 单次插入的交换成本为 $O(k)$; 配合随机深度选择, 平均探测复杂度可控制在 $O(\log^{(k)} n)$ 量级, 从而与上述描述的空间一时间权衡曲线对齐^[9-10,20]。

2.5.2 MiniArray M

k-kick 的 IsSaturated、GetMaxUsableDepth 与 ReInsert 都需要在 bin 上判断是否存在空位并找到可用位置。MiniArray M 为各深度 bin 维护等价于 bitmap 的空闲槽位图^[28], 使上述判定与定位在均摊意义下为常数时间, 而不必扫描整个 Cubby^[4,20]。MiniArray 则通过 $O(1)$ 深度 B 树、叶内 Rank/Select 与 Packed Leaf 维护变长元数据, 并实现常数时间复杂度的 Access/Insert/Delete/Update 函数^[24-26]; 树高 $O(1)$, 内部指针 $\Theta(\log \log n)$ 比特^[12,15]。

2.5.3 Local Query Router: $O(1)$ 查询与映射存储

偏好序列与探测位置序列在插入阶段完整定义; 为实现 $O(1)$ 查询, 须额外维护键到探测下标的映射, 而不遍历 $h_1(x), h_2(x), \dots$: k-kick 插入会搬移元素, 其实际位置既不等于首选位置 $g_k(x)$, 也不等于 $h_1(x)$ 。Local Query Router 因此为每个键保存映射

$$\pi(x) \longrightarrow j,$$

其中 j 为探测序列下标。Cubby 内令 $g_I(x) = g^*(x)[1, \log |I|]$, 所有满足 $g_I(x) = i$ 的键由 router $A[i]$ 管理; $A[i]$ 实现为二叉前缀树 (可进一步 Patricia 压缩^[14,29]), 对外接口为 $\text{insert}(\text{key}, j)$ 、 $\text{query}(\text{key})$ 、 $\text{delete}(\text{key})$ 。查询时先算 $i = g_I(x)$, 在 $A[i]$ 得 j , 再访问 $\text{storage}[h_j(x)]$, 加上 $B[h_j(x)]$ 与 payload 恢复 $\pi(x)$, 与查询侧的 $\pi(x)$

进行对比； $j \leq \log K$ ，每条映射仅需 $O(\log \log n)$ 比特，查询路径为常数次 router 与 MiniArray 访问^[4,8]。

2.5.4 辅助结构同步不变量

元素在 k-kick 踢出链中每发生一次搬移，须同步更新四处状态：

- storage 槽位内容与 insert_depth；
- $B[i]$ 中的 δ 与中间位；
- $A[g_I(x)]$ 中 $\pi(x) \mapsto j$ 的映射；
- M 中各 bin 的空闲位图。

删除非 tail 元素时的 tail 随机回填、tier 合并/拆分后的全量重插，以及表级双表扩缩容，均遵循同一同步规则——不可以仅复制 payload 而忽略 router 与 B 的上下文依赖^[2,4]。第三章将给出 MiniArray、k-kick 与 Local Query Router 的全部算法伪代码及统一操作语义。

2.6 本章小结

本章建立了后文所需的理论语境：球槽分配模型给出探测复杂度与交换成本的定义^[1,17]；全域参数与 $g^*(x)$ 位段划分明确了地址语义；图 2-2 与 Facility-Cubby-Slot(storage/ $A/M/B$) 层级关系给出整体骨架；商压缩说明 payload 与 $B[i]$ 的信息分工^[4,14]；同时从 k-kick 交换上界、MiniArray M 的空闲判定、Local Query Router 的 $O(1)$ 查询及四个状态同步不变量四个角度，为第三章算法设计提供理论锚点。下一章将给出各模块的完整伪代码；第四章则报告宏基准与参数扫描下的性能测量。

第三章 哈希表结构与算法设计

本章依照系统核心设定，给出分层哈希表各模块组件的完整结构与算法细节。内容覆盖 Facility、MiniArray、多层级 Cubby、Cubby 内 k-kick 与 Local Query Router，商压缩的实现，并在最后一节统一说明插入、查询与删除的操作语义。

3.1 Facility 与 MiniArray

3.1.1 Facility 的基本属性

每个 Facility 对应全局表中的一个局部片段，规模为 $K = \text{polylog } N$ 。Facility 管理其内部的全部 Cubby（包括各 tier 及唯一 tail），并维护一个 Facility 层次的 MiniArray，用于定位键对应的 Cubby。B 树的 fanout(上文有定义，fanout 为每个内部节点的子指针个数) 记为 F ，取 $F = \Theta(\log n / \log \log n)$ ；树高为 $O(1)$ ，每个内部节点的子指针仅需 $\Theta(\log \log n)$ 比特^[4,24-25]。

3.1.2 MiniArray 的逻辑结构

MiniArray 由以下部分共同组成：

MiniArray $\rightarrow O(1)$ 深度 B 树 $\rightarrow$ Packed Leaf + Bitmap + Rank/Select + Lookup Table

叶节点 Leaf 包含：

- bitmap：用于标记叶内哪些逻辑位置已有有效元素，其大小与叶容量相同；
- data：PackedArray，在有效位置上紧凑、无间隙地存放变长元素。

内部节点 Internal 包含：

- subtree_size[F]：每个子树中有效元素总数，供 Rank/Select 定位；
- children[F]：至多 F 个子女指针，每个指针 $\Theta(\log \log n)$ 比特。

MiniArray 对外提供的接口函数为 Access(idx)、Insert(idx, val)、Delete(idx)、Update(idx, val)，其中 idx 为逻辑下标。叶容量上限为 MAX_LEAF_SIZE，由 NODE_MAX_BITS 常数 $c \in [1, 3]$ 产生^[4,12,15]。

3.1.3 Access：按逻辑下标读取

Access 操作的输入是逻辑下标，搜索过程自根向下逐层向下遍历，利用各层 subtree_size 进行前缀和判断，将输入的逻辑下标映射到对应的目标叶节点；在叶子节点的内部用 Select 在 bitmap 上定位第 idx 个有效位置，再从 data 读出元素。树高为 $O(1)$ ，叶内 Select 算法通过 Lookup Table 常数时间完成^[8,15,26]，不需要遍历比特串，获取到物理位置之后，直接从紧凑数组中读取对应的元素，并返回。

算法 1 MiniArray.Access

```

1: 输入逻辑下标 idx
2: cur ← root, rem ← idx
3: while cur 为内部节点 do
4:   找到子树  $i$  使  $\sum_{t < i} subtree\_size[t] \leq rem < \sum_{t \leq i} subtree\_size[t]$ 
5:   rem ← rem -  $\sum_{t < i} subtree\_size[t]$ 
6:   cur ← cur.child[i]
7: end while
8: pos ← CallSelect(cur.bitmap, rem)
9: 返回 cur.data[pos]
```

3.1.4 Insert：插入与叶分裂

插入过程中，Insert 在定位叶节点的过程中维护路径栈，为后续自底向上更新 subtree_size 提供途径；在叶节点内用 Select 函数找到插入位置，将 bitmap 对应位置赋值为 1，并向 data 写入 val。若叶内有效元素数达到叶子容量的上限，就会调用 Split 对叶子节点进行分裂操作；插入与分裂检查完成之后，自底向上对路径栈中每个内部节点的 subtree_size 加一，以保证后续查询与定位的正确性。

算法 2 MiniArray.Insert

```
1: 输入 idx, val; 初始化空路径栈 stk
2: cur ← root, rem ← idx
3: while cur 为内部节点 do
4:   stk.push(cur)
5:   找到子树  $i$  使  $\sum_{t < i} subtree\_size[t] \leq rem < \sum_{t \leq i} subtree\_size[t]$ 
6:   rem ← rem -  $\sum_{t < i} subtree\_size[t]$ 
7:   cur ← cur.child[i]
8: end while
9: leaf ← cur
10: pos ← CallSelect(leaf.bitmap, rem)
11: leaf.bitmap.set(pos); leaf.data.insert(pos, val)
12: if leaf.size ≥ MAX_LEAF_SIZE then
13:   CallSplit(leaf)
14: end if
15: for 路径栈中每个内部节点 node do
16:   node.subtree_size[i] += 1
17: end for
```

3.1.5 Delete: 删除与叶合并

Delete 同样沿 subtree_size 定位叶节点, 用 Select 函数在 Bitmap 上找到待删元素的具体位置, 清除 bitmap 对应位, 并从 data 移除对应的元素。若叶内有效元素过少, 低于阈值, 调用 Merge 与相邻兄弟叶合并, 合并可能需要多次操作才能保证 B 树恢复数据结构; 最后自底向上将路径上 subtree_size 减一。

算法 3 MiniArray.Delete

```
1: 输入 idx; 初始化路径栈 stk
2: 沿 subtree_size 定位至叶 leaf(同 Insert)
3: pos ← CallSelect(leaf.bitmap, rem)
4: leaf.bitmap.unset(pos); leaf.data.remove(pos)
5: if leaf 有效元素过少 (即 leaf.size < MAX_LEAF_SIZE / 2) then
6:   CallMerge(leaf)
7: end if
8: for 路径栈中每个内部节点 node do
9:   node.subtree_size[i] -= 1
10: end for
```

3.1.6 Update: 原地覆写

Update 仅修改已有有效位置的元素值, 不改变 bitmap 中 1 的个数, 也不触发 Split/Merge。定位过程与 Access 相同, 在叶内 Select 后直接 $\text{leaf.data}[\text{pos}] \leftarrow \text{val}$ 。

算法 4 MiniArray.Update

- 1: 输入 idx, val
 - 2: 沿 subtree_size 定位至叶 leaf
 - 3: $\text{pos} \leftarrow \text{CallSelect}(\text{leaf.bitmap}, \text{rem})$
 - 4: $\text{leaf.data}[\text{pos}] \leftarrow \text{val}$
-

3.1.7 Split 与 Merge

Split (叶分裂)。新建叶节点, 以 $\text{leaf.size}/2$ 为界将后半段 bitmap 与 data 迁至新叶, 原叶保留前半段; 在父节点 child 数组中插入新叶指针, 并更新 subtree_size 的分裂计数。

算法 5 MiniArray.Split (叶分裂)

- 1: 输入叶 leaf
 - 2: $\text{new_leaf} \leftarrow$ 新建 LeafNode; $\text{mid} \leftarrow \text{leaf.size}/2$
 - 3: 将 leaf.bitmap 、 leaf.data 后半段搬至 new_leaf
 - 4: leaf 保留前半段
 - 5: 在父节点 parent.child 中插入 new_leaf , 更新 $\text{parent.subtree_size}$
-

Merge (叶合并)。取相邻兄弟叶, 将兄弟的 bitmap 与 data 并入当前叶, 从父节点删去兄弟子树并刷新 subtree_size 。

算法 6 MiniArray.Merge (叶合并)

- 1: 输入叶 leaf ; $\text{sib} \leftarrow$ 相邻兄弟叶; $\text{parent} \leftarrow \text{leaf}$ 的父节点
 - 2: $\text{leaf.bitmap} \mid = \text{sib.bitmap}$; $\text{leaf.data.append}(\text{sib.data})$
 - 3: 从 parent.child 移除 sib , 更新 $\text{parent.subtree_size}$
-

3.1.8 Rank 与 Select

叶子节点中 bitmap 上的 Rank/Select 通过 Lookup Table 实现, 这是紧凑动态结构的标准组件^[4,8,11,26]。

- $\text{Rank}(\text{bitmap}, i)$: 返回 bitmap 前 i 位中 1 的个数，等于 LookupTable 对高段与低段的查表结果之和；
- $\text{Select}(\text{bitmap}, k)$: 返回 bitmap 中第 k 个 1 的位置，由 $\text{LookupTable}[k]$ 直接给出。

算法 7 Rank / Select（查表实现）

```

1: function RANK(bmp, i)
2:   返回  $\text{LookupTable}[\text{bmp 高段}] + \text{LookupTable}[\text{bmp 低段}]$ 
3: end function
4: function SELECT(bmp, k)
5:   返回  $\text{LookupTable}[k]$  ▷ 第  $k$  个 1 的位置
6: end function

```

3.1.9 复杂度与空间性质

Facility 级 MiniArray 的树高为 $O(1)$ ，自根至叶的每层路由仅依赖 subtree_size 比较，叶内 Rank/Select 经 Lookup Table 常数时间完成^[8,26]。因此 Access 与 Update 为最坏情况 $O(1)$ ；Insert 与 Delete 在叶分裂/合并均摊意义下亦为 $O(1)$ ^[4,24]。空间方面，每个内部节点的 F 个子指针各需 $\Theta(\log \log n)$ 比特，叶节点以 bitmap 标记有效位置、以 PackedArray 无间隙存放变长元素，整体辅助位数与 $O(\log^{(k)} n)$ 的全局预算相容^[12,15]。Cubby 内的 A 、 M 、 B 三组 MiniArray 复用同一套接口语义： $A[i]$ 存 router 映射， M 存 bin 空闲图， B 与 storage 槽位一一对应存差分编码——三者在 k-kick 搬移时须原子同步更新。

3.2 多层次 Cubby 设计

3.2.1 层级属性与初始状态

Facility 内 Cubby 按 tier 分级管理。 j -tiered Cubby 的目标数量为

$$t_j = \Theta\left(\frac{(\log^{(j+1)} n)^2}{(\log^{(j)} n)^2}\right),$$

容量（槽位数）为

$$r_j = \frac{K}{(\log^{(j)} n)^2}.$$

系统初始时，每个 Facility 仅包含一个 1-tiered Cubby，且该 Cubby 即为 tail。此后所有新键插入 tail；tail 满后，旧 tail 被视为一个普通的 1-tiered Cubby 挂入 tier 集合，并新建 tail 承接后续插入，该过程可能触发 tier 向上合并^[2,4,6-7]。

3.2.2 动态调整：合并与拆分

设当前 j -tiered Cubby 的实际个数为 cnt_j 。调度规则如下：

- 当 $\text{cnt}_j \geq 3t_j$ 时，取出 t_j 个 j -tiered Cubby，将其内全部元素导出，合并为一个 $(j+1)$ -tiered Cubby 并重新插入；
- 当 $\text{cnt}_j \leq t_j/2$ 且存在非空的 $(j+1)$ -tiered Cubby 时，取出一个 $(j+1)$ -tiered Cubby，拆成 t_j 个 j -tiered Cubby 并重插元素；
- 无论合并或拆分，均须取出 cubby 内所有元素重新插入，并同步更新对应 Cubby 中的 MiniArray A 、 M 、 B 与 storage，不能仅复制旧槽位内容（因 r_j 、 $|I|$ 与商压缩上下文可能已变）^[4-5,27]。

算法 8 j -tiered cubby 数量调度

```

1: for 每个 tier 层级  $j$  do
2:   计算目标数量  $t_j$ ，统计实际数量  $\text{cnt}_j$ 
3:   if  $\text{cnt}_j \geq 3t_j$  then
4:     取出  $t_j$  个  $j$ -tiered Cubby，合并为一个  $(j+1)$ -tiered Cubby 并重新插入
5:   else if  $\text{cnt}_j \leq t_j/2$  且 tier  $j+1$  非空 then
6:     取出一个  $(j+1)$ -tiered Cubby，拆成  $t_j$  个  $j$ -tiered Cubby 并重新插入
7:   end if
8: end for

```

3.2.3 Cubby 内部组件

每个容量为 $|I|$ 的 Cubby 包含以下五部分：

storage 数组。长度为 $|I|$ ，存放键值的商压缩载荷；槽位 i 上的 $\text{storage}[i]$ 对应 $\pi(x)[\log N + 1, \log U]$ 的 payload 部分，完整键恢复依赖 $B[i]$ 与全局上下文。

k-kick 插入逻辑。在 storage 上按一定的规则动态决定元素实际槽位，并维护每个占用元素的 insert_depth。

MiniArray A 。长度为 $|I|$ 的 Local Query Router 数组； $A[i]$ 是一棵二叉前缀树，管理所有满足 $g_I(x) = i$ 的键。

MiniArray M 。存储各 k -kick 深度 bin 的空闲槽位图，使插入、删除与 ReInsert 能在常数时间内判定 bin 是否含空位、定位空位，而不必扫描整个 Cubby。

MiniArray B 。与 storage 槽位一一对应。当 $\text{storage}[i]$ 存放键 x 的信息时， $B[i]$ 保存

$$\delta = g_I(x) - i, \quad g_I(x) = g^*(x)[\log |I| + 1, \log K],$$

以及 $g^*(x)[\log |I| + 1, \log K]$ 的中间位。配合 Facility 编号 $r = g^*(x)[\log K + 1, \log N]$ 与 payload，可完整恢复 $\pi(x)^{[4,8]}$ 。

3.2.4 插入与删除操作

插入。每次插入均进入当前 Facility 的 tail Cubby。若 tail 仍有空位，则在 tail 内执行 k -kick 插入；若 tail 已满，则新建 tail，并将已满的旧 tail 视作 1-tiered Cubby 挂入 1-tier Cubby 集合，该步骤可能因 $\text{cnt}_1 \geq 3t_1$ 而触发向上合并。

删除与 tail 回填。若删除发生在 tail 之外的其他 Cubby，删除完成后从 tail 中随机选取一个占用元素，经 k -kick 规则回填至刚空出的槽位，并更新 MiniArray A 、 M 、 B 与 storage。若 tail 在回填后为空，则从 1-tiered 层取出一个 Cubby 提升为 tail；该过程可能因 $\text{cnt}_i \leq t_i$ 而触发向下拆分（甚至可能会触发多层）。上述不变量保证：除 tail 外，各 Cubby 尽量维持满载，局部波动由 tail 与 tier 调度共同吸收^[4,17-18]。

算法 9 tail 维护与删除回填

- 1: 插入：写入 tail；若 tail 满则旧 tail 归入 1-tiered Cubby 并新建 tail
 - 2: 删除（非 tail）：释放被删槽位；从 tail 随机取元素 y 回填
 - 3: 对 y 执行 k -kick 写入，更新 A, M, B
 - 4: **if** tail 为空 **then**
 - 5: 从 1-tiered Cubby 提升一个 Cubby 为 tail；必要时触发 tier 拆分
 - 6: **end if**
-

3.3 k -kick 层级插入算法

约定：Cubby 内算法中的 x 均指 $\pi(x)$ （置换后的值），而非原始键；符号仍记为 x ，设 Cubby 容量为 $|I|$ 。

3.3.1 层级 bin 定义

k-kick 不对 storage 做物理分层，仅在逻辑上将 Cubby 划分为 $k + 1$ 个深度， k 为常数（通常 $k \in [3, 5]$ ，基准取 4），定义不同层次的 bin：

- 深度 0：唯一 bin 覆盖整个 Cubby，大小 $s_0 = K = \text{polylog } n$ ；
- 深度 1：将深度 0 的 bin 均匀切分为许多大小 $s_1 = \Theta((\log K)^6)$ 的子 bin；
- ...
- 深度 i ($i \geq 1$)：bin 大小 $s_i = \Theta((\log^{(i)} K)^6)$ ，由上一层 bin 继续均匀切分得到。

定义函数 $\text{Bin}(x, i)$ 返回键 x 在深度 i 所属 bin 的槽位区间：深度 0 时返回 $[0, |I| - 1]$ ；深度 $i \geq 1$ 时在 $\text{Bin}(x, i - 1)$ 内按 s_i 均匀切分，并根据 x 的哈希比特在每一层中选定对应的一个子区间^[4,22]。

3.3.2 偏好序列

对键 x 定义偏好序列

$$g_0(x) \rightarrow g_1(x) \rightarrow \cdots \rightarrow g_k(x).$$

其中 $g_0(x) = \text{Bin}(x, 0)$ 为整个 Cubby； $g_{i+1}(x)$ 为 $g_i(x)$ 经均匀切分后 x 所落子 bin。于是 $g_{i+1}(x)$ 一定是 $g_i(x)$ 的子 bin；沿路径自顶向下，每一层均为上一层的随机子区间。自底向上回溯 $g_k, g_{k-1}, \dots, g_0$ 即可得到全部祖先 bin。

算法 10 PreferenceSequence: 偏好序列

```

1: function PREFERENCESEQUENCE( $x$ )
2:    $g[0] \leftarrow \text{BIN}(x, 0)$ 
3:   for  $i = 1$  to  $k$  do
4:      $g[i] \leftarrow g[i - 1]$  内均匀切分后、 $x$  所属的子 bin
5:   end for
6:   返回  $g[0..k]$ 
7: end function

```

3.3.3 探测位置序列

$h_j(x)$ 表示 x 在 Cubby storage 中的第 j 个候选槽位。序列按先深后浅顺序构造：依次遍历 $g_k(x), g_{k-1}(x), \dots, g_0(x)$ ，并在每个 bin 内按槽位顺序枚举全部位置。

对深度 i 的 bin $g_i(x)$ ，设

$$\text{base}_i = s_k + s_{k-1} + \dots + s_{i+1},$$

则对任意 $j \in [0, s_i - 1]$ ，全局探测序列中对应项为

$$h_{\text{base}_i+j}(x) = g_i(x) \text{ 的第 } (j+1) \text{ 个槽位.}$$

注意文档中下标公式 $h_{(k+1-i) \cdot s_i + j}(x)$ 与 base_i 表述等价：更深层的 bin 在序列中排在更前。序列总长度约为 $\sum_{i=0}^k s_i$ ；查询阶段并不线性扫描该序列，而由 Local Query Router 直接给出实际使用的下标 $j^{[4,20]}$ 。

算法 11 ProbeSequence: 探测位置序列（先深后浅）

```

1: function PROBESEQUENCE( $x$ )
2:    $g \leftarrow \text{PREFERENCESEQUENCE}(x)$ ;  $seq \leftarrow []$ 
3:   for  $i = k$  downto 0 do
4:      $bin \leftarrow g[i]$ 
5:     for  $pos$  从  $bin.start$  到  $bin.end$  do
6:        $seq.append(pos)$ 
7:     end for
8:   end for
9:   返回  $seq$ 
10: end function

```

$\triangleright h_1(x), h_2(x), \dots$ 依次为 $seq[0], seq[1], \dots$

3.3.4 bin 饱和与最大可用深度

在深度 d 下，称 bin 饱和，当且仅当 bin 已满，且其中每个元素 y 均满足 $y.insert_depth \geq d$ 。bin 未饱和时不饱和；bin 已满但存在 $y.insert_depth < d$ 的元素时，亦不视为饱和。

GetMaxUsableDepth(x) 自 k 向 0 扫描偏好序列，返回最深的饱和 bin 的深度；若全部饱和则返回 0(说明已满)。

算法 12 IsSaturated 与 GetMaxUsableDepth

```
1: function ISSATURATED(bin,  $d$ )
2:   if bin 未滿 then
3:     返回 false
4:   end if
5:   for bin 中每个元素  $y$  do
6:     if  $y.\text{insert\_depth} < d$  then
7:       返回 false
8:     end if
9:   end for
10:  返回 true
11: end function
12: function GETMAXUSABLEDEPTH( $x$ )
13:   $g \leftarrow \text{PREFERENCESEQUENCE}(x)$ 
14:  for  $d = k$  downto 0 do
15:    if not ISSATURATED( $g[d]$ ,  $d$ ) then
16:      返回  $d$ 
17:    end if
18:  end for
19:  返回 0
20: end function
```

3.3.5 随机深度选择与 InsertKick

插入时须记录 insert_depth 并在各深度 bin 间均匀分布。插入前先取随机哈希 $s(x) \in [0, k]$, 再令

$$\text{depth} = \min(\text{GETMAXUSABLEDEPTH}(x), s(x)).$$

该式为随机深度与可用深度的 min 组合, 对贪心上界形成软上限, 使高深度 bin 更长时间保持未饱和。在目标 bin $g_{\text{depth}}(x)$ 中: 若有空位, 则写入 x 、记录 $x.\text{insert_depth} = \text{depth}$, 并在 Local Query Router $A[g_I(x)]$ 中登记探测下标 j ; 若无空位, 则踢出 bin 内 insert_depth 最小的元素 y , 将 x 写入被踢槽位, 再对 y 调用 ReInsert。整条踢出链至多触发 k 次, 与 $O(k)$ 交换上界一致^[4]。

3.3.6 ReInsert: 被踢元素向上放入

插入发生踢出时, 被踢元素须安置到合适深度; **ReInsert** 自 $y.\text{insert_depth}-1$ 向上逐层检查 $\text{Bin}(y, d)$: 若该 bin 在深度 d 下不饱和且有空位, 或存在 insert_depth 小于 d 的元素, 则将 y 写入并可能继续踢出链式调用 ReInsert。深度降至 0 仍无

算法 13 InsertKick: k-kick 插入

```
1: 输入  $x$  (已为  $\pi(x)$ )
2:  $depth\_max \leftarrow \text{GETMAXUSABLEDEPTH}(x)$ 
3:  $s_x \leftarrow \text{RANDOMHASH}(x) \bmod (k + 1)$ 
4:  $depth \leftarrow \min(depth\_max, s_x)$ 
5:  $g \leftarrow \text{PREFERENCESEQUENCE}(x)$ ;  $target \leftarrow g[depth]$ 
6: if  $target$  有空位 (由  $M$  判定) then
7:   在任意空位写入  $x$ ;  $x.insert\_depth \leftarrow depth$ 
8:   更新  $A, M, B$ ; 返回成功
9: else
10:   $y \leftarrow target$  中  $insert\_depth$  最小元素;  $evict\_pos \leftarrow y$  的位置
11:  从  $evict\_pos$  移除  $y$ ; 将  $x$  写入  $evict\_pos$ 
12:   $x.insert\_depth \leftarrow depth$ 
13:   $\text{REINSERT}(y)$ 
14: end if
```

法安置时 Cubby 已满, 须新建 tail Cubby 并重试 InsertKick, 与多层级 Cubby 设计衔接。MiniArray M 在此过程中的空位判定与定位均为常数时间。理论保证: 在参数满足 Bender 等人^[4] 的设定时, 极大概率至多 k 次 kick 后一定能安置被踢元素, 如算法 13 所示。

算法 14 ReInsert: 被踢元素向上找空位

```
1: function  $\text{REINSERT}(y)$ 
2:   for  $d = y.insert\_depth - 1$  downto 0 do
3:      $bin \leftarrow \text{BIN}(y, d)$ 
4:     if  $\text{ISSATURATED}(bin, d)$  then
5:       continue
6:     end if
7:     if  $bin$  有空位 (由  $M$  判定) then
8:       将  $y$  写入空位;  $y.insert\_depth \leftarrow d$ 
9:       更新  $A, M, B$ ; 返回成功
10:    else if  $bin$  中存在  $z.insert\_depth < d$  then
11:       $z \leftarrow bin$  中  $insert\_depth$  最小且满足  $z.insert\_depth < d$  的元素
12:      从  $bin$  移除  $z$ ; 将  $y$  写入该槽位
13:       $y.insert\_depth \leftarrow d$ ; 更新  $A, M, B$ 
14:       $\text{REINSERT}(z)$ ; 返回
15:    end if
16:  end for
17:  返回失败 ▷ 深度已降至 0 仍无法安置, Cubby 已满
18:  新建 tail cubby; 在新 tail 中对  $y$  重试 INSERTKICK
19: end function
```

3.3.7 复杂度与正确性

单次 InsertKick 至多触发一条长度为 k 的踢出链：每次踢出仅调用一次 ReInsert，而被踢元素的 insert_depth 严格下降，故链长受 k 约束，与 Bender 等人^[4]的 $O(k)$ 交换成本一致。

GetMaxUsableDepth 自 k 向 0 扫描，返回最深的饱和 bin；随机深度 $\text{depth} = \min(\text{depth_max}, s(x))$ 在理论上保证：若存在可写 bin，则插入不会触发深层踢出，同时使不同深度的 bin 负载趋于均衡^[19]。

bin 中的是否存在空位与定位空位都通过 MiniArray M 完成，避免对整个 Cubby 进行依次遍历 $O(|I|)$ 时间复杂度的扫描；ReInsert 从 $y.\text{insert_depth}-1$ 向上逐层检查，每层仅访问 M 与局部 bin 状态，均摊常数时间。

元素每次写入或迁移，都需要同步更新 storage、insert_depth、 B 、 $g_I(x)$ 中的 $\pi(x) \mapsto j$ 映射对应的信息，以及 M 的空闲位图，这一同步规则是整个系统正常运行的前提，之后的操作才能正常运行。

3.4 Local Query Router

3.4.1 设计目标

偏好序列与探测位置序列 $h_1(x), h_2(x), \dots$ 在插入分析中完整定义了候选槽位顺序。然而 k-kick 插入会通过踢出动态挪动元素：元素实际位置一般不等于首选深度 $g_k(x)$ ，也不等于 $h_1(x)$ 。若查询仍沿 $h_1, h_2, \dots$ 线性扫描，时间复杂度将与探测序列长度同阶，无法达到 $O(1)$ 。因此必须为每个键保存精准映射

$$\pi(x) \longrightarrow j,$$

其中 j 为探测序列中的下标，查询时直接访问下标 $h_j(x)$ 对应的 storage 中的数据，再用 B 与 payload 三个数据恢复 $\pi(x)$ 后与查询元素进行对比校验^[4,8]。

3.4.2 映射规则

Cubby 内定义局部首选槽位号（与 k-kick 偏好序列无关）：

$$g_I(x) = g^*(x)[1, \log |I|] \in [0, |I| - 1].$$

所有满足 $g_I(x) = i$ 的键由第 i 号 Local Query Router $A[i]$ 管理。查询时先计算 $i = g_I(x)$ ，再在 $A[i]$ 中查询 $\pi(x)$ 得到探测序列下标 j （若不存在则未命中）；随后访问 storage 中 $h_j(x)$ 对应槽位，读取 payload 与 B ，重建 $g^*(x)$ 并校验是否命中。

3.4.3 二叉前缀树存储与接口

每个 $A[i]$ 实现为一棵二叉前缀树；对外键为 $\pi(x)$ 的比特串。接口为：

- `insert(key, j)`: $j \leq \log K$ ，仅需 $O(\log \log n)$ 比特存储；如算法 15 所示；
- `query(key)`: 返回 j 或 “不存在”；如算法 16 所示；
- `delete(key)`: 删除映射。如算法 17 所示。

树中不存完整键，仅存区分键的最短前缀；叶节点存 `j_value`。

插入过程利用映射后的 $\pi(x)$ 进行操作；下文以 x 表示 $\pi(x)$ 的比特串。映射后的 x 的每一位（从高位到低位）依次决定在树中向左（0）还是向右（1）走。沿路径向下，若遇空子树则创建新节点；到达叶节点时标记为叶并存 `j_value`。即使管理的键数较多时，树高仍受限于键的位数，故访问步数为 $O(\log U)$ 。

3.4.4 复杂度与 Patricia 压缩

对键长 $\log U = \Theta(\log n)$ 的比特串，`query` 与 `insert` 沿树向下至多 $\log U$ 步；Patricia 风格的前缀压缩^[8,14,29] 将仅有一位不同的键合并为同一压缩边，使实测 router 深度（`router_steps_max`）远小于键长上界。每条映射存 $j \leq \log K = O(\log \log n)$ 比特，与整体 $O(\log^{(k)} n)$ 空间目标一致^[4-5]。

动态维护。k-kick 踢出链中，被搬移键须 `delete` 旧映射再 `insert` 新 j ；删除操作清空叶 `j_value` 但不破坏共享前缀路径上的内部节点，均摊开销与 $A[i]$ 所管理

算法 15 Local Query Router.insert

```
1: function INSERT(key, j)
2:   cur  $\leftarrow$  root
3:   for key 的每一位 bit do
4:     if bit = 0 then
5:       if cur.child0 为空 then 创建子节点
6:       end if
7:       cur  $\leftarrow$  cur.child0
8:     else
9:       if cur.child1 为空 then 创建子节点
10:      end if
11:      cur  $\leftarrow$  cur.child1
12:    end if
13:  end for
14:  cur.is_leaf  $\leftarrow$  true; cur.j_value  $\leftarrow$  j
15: end function
```

算法 16 Local Query Router.query

```
1: function QUERY(key)
2:   cur  $\leftarrow$  root
3:   for key 的每一位 bit do
4:     if bit = 0 then
5:       if cur.child0 为空 then 返回 NOT_FOUND
6:       end if
7:       cur  $\leftarrow$  cur.child0
8:     else
9:       if cur.child1 为空 then 返回 NOT_FOUND
10:      end if
11:      cur  $\leftarrow$  cur.child1
12:    end if
13:  end for
14:  if cur.is_leaf then 返回 cur.j_value
15:  else 返回 NOT_FOUND
16:  end if
17: end function
```

算法 17 Local Query Router.delete

```
1: function DELETE(key)
2:   沿 key 的比特路径找到叶节点
3:   将叶标记为非叶 (is_leaf  $\leftarrow$  false), 清除 j_value
4: end function
```

的局部键数相关。在 $K = \text{polylog } n$ 且 Cubby 容量 $|I| \leq K$ 的设定下，单个 $A[i]$ 的规模受控，router 更新与查询均摊为常数时间^[2,4]。

3.5 商压缩与键恢复

3.5.1 设计动机与信息分工

当前层次结构中，查询路由需用映射后二进制低位定位 Facility 与 Cubby 内数组；若槽位仍存完整键，则寻址路径上的信息上下文被重复占用。商压缩借助随机可逆置换 π ，将键映射为 $\pi(x)$ ，并按下述原则拆分信息： $\pi(x)$ 的低 $\log N$ 位 $g^*(x) = \pi(x)[1, \log N]$ 的各段位由全局路由与局部地址上下文（Facility 编号、Cubby 容量、实际槽位下标）及 MiniArray B 中的差分元数据共同承担；槽位 storage 仅保存无法由上述上下文直接推出的高位商，即 payload

$$\text{payload} = \pi(x)[\log N + 1, \log U].$$

查询时 Local Query Router 给出探测位置序列 j ， $h_j(x)$ 定位到 storage 中的具体位置，然后从 $\text{storage}[h_j(x)]$ 、 $B[h_j(x)]$ 与 Facility/Cubby 上下文联立恢复完整 $\pi(x)$ ，再经 π^{-1} 得到原始键。该分工使每个元素在槽位层面仅占用 $\log U - \log N$ 比特的有效载荷，而将路由相关的 $\log N$ 位分散到地址结构与变长元数据中，与整体 $O(\log^{(k)} n)$ 空间目标一致^[4,8]。

3.5.2 $\pi(x)$ 与 $g^*(x)$ 的位段划分

记 $x[a, b]$ 为 x 自第 a 位至第 b 位（自低位向高位）组成的子串。对键 x 经置换得 $\pi(x)$ ，全局路由哈希为

$$g^*(x) = \pi(x)[1, \log N].$$

在 $g^*(x)$ 内部，自高向低继续划分为三段（见图 2-1，第二章已给出示意）：

- Facility 编号段： $r = g^*(x)[\log K + 1, \log N] = \lfloor g^*(x)/K \rfloor$ ，用于定位键所属 Facility；

- Cubby 中间段: $g^*(x)[\log |I| + 1, \log K]$, 用于在 Facility 内区分不同 Cubby 及局部桶范围;
- Local Query Router 段: $g_I(x) = g^*(x)[1, \log |I|] \in [0, |I| - 1]$, 与 k-kick 偏好序列无关, 仅决定键归属 $A[i]$ 的桶号 i 。

k-kick 插入可能将元素 x 写入槽位 $i \neq g_I(x)$ 的实际位置。此时 $g_I(x)$ 无法由槽位下标 i 直接读出, 须额外保存差分

$$\delta = g_I(x) - i,$$

存入 MiniArray $B[i]$ 。

3.5.3 MiniArray B 中的 MetaEntry

每个占用槽位 i 对应一条变长元数据 MetaEntry, 经 MiniArray B 的 Access/Update 接口读写。逻辑字段如下:

- delta: 有符号整数 $\delta = g_I(x) - i$, 编码为 zigzag varint, 适应 k-kick 搬移后槽位与 router 桶号的不一致;
- mid_bits: $g^*(x)$ 在 Cubby 范围内的中间位, 即 $g_k = g^*(x) \bmod K$ 满足 $g_k = \text{mid_bits} \cdot |I| + g_I(x)$ 的商部分 $\lfloor g_k / |I| \rfloor$; 位宽 $\text{mid_w} = \lceil \log_2((K-1)/|I|+1) \rceil$, 由 $(K, |I|)$ 派生;

编码顺序为: 先写入 delta 的 varint, 再写入 mid_bits 的固定位宽字段; 解码时按相同布局与 MetaCodecLayout 逆序读出。 $B[i]$ 的 bitlen 随条目实际长度变化, 由 Facility 级 MiniArray 的 Rank/Select 机制定位, 与第二章所述变长元数据维护方式一致。

3.5.4 恢复：重建 $\pi(x)$

给定 Facility 编号 r 、Cubby 容量 $|I|$ 、槽位下标 i 、 $\text{storage}[i]$ 中的 payload 及 $B[i]$ 解码得到的 MetaEntry，按下式自底向上重建 $g^*(x)$ 与 $\pi(x)$ ：

$$g_I(x) = i + \delta, \quad (3-1)$$

$$g_k = \text{mid_bits} \cdot |I| + g_I(x), \quad (3-2)$$

$$g^*(x) = r \cdot K + g_k, \quad (3-3)$$

$$\pi(x) = (\text{payload} \ll \log N) \mid g^*(x). \quad (3-4)$$

各步均须校验 $g_I(x) \in [0, |I|)$ 、 $g_k < K$ 及 $g^*(x) < N$ ；任一不满足则恢复失败，视为未命中。原始键由 $x = \pi^{-1}(\pi(x))$ 得到。

算法 18 ReconstructGxPi: 从槽位上下文恢复 $\pi(x)$

```

1: function RECONSTRUCTGxPi( $r, N, K, |I|, \text{slot}, \text{payload}, \text{meta}, \log N$ )
2:    $g_I \leftarrow \text{slot} + \text{meta}.\text{delta}$ 
3:   if  $g_I \notin [0, |I|)$  then
4:     返回失败
5:   end if
6:    $g_k \leftarrow \text{meta}.\text{mid\_bits} \cdot |I| + g_I$ 
7:   if  $g_k \geq K$  then
8:     返回失败
9:   end if
10:   $g_{\text{star}} \leftarrow r \cdot K + g_k$ 
11:  if  $g_{\text{star}} \geq N$  then
12:    返回失败
13:  end if
14:  返回  $(\text{payload} \ll \log N) \mid g_{\text{star}}$ 
15: end function

```

3.5.5 查询校验

查询键 x 时，Router 给出探测下标 j ，访问候选槽位 $i = h_j(x)$ 。只有当完整恢复的值与查询的值相同，才认为查询命中：

Router 仅负责将查询映射到 $O(I)$ 个候选位置；最终是否命中必须由 payload、 B 与 $(r, N, K, |I|)$ 联立恢复的 $\pi(x)$ 与查询侧计算的 $\pi(x)$ 一致方可确认。该“Router 定位 + 商压缩校验”的两段式路径，是系统能够不存储完整键的前

算法 19 SlotPiMatches: 槽位命中校验

```
1: function SLOTPIMATCHES(cubby, slot,  $r$ , table, query_gx)
2:   if cubby.slots[slot] 为空 then
3:     返回 false
4:   end if
5:   meta  $\leftarrow$  DECODEMETAENTRY(cubby.B[slot])
6:   gx  $\leftarrow$  RECONSTRUCTGxPi( $r$ ,  $N$ ,  $K$ ,  $|I|$ , slot, payload, meta, logN)
7:   if gx 失败 then
8:     返回 false
9:   end if
10:  返回 gx = query_gx
11: end function
```

提^[4,8]。

3.5.6 空间性质

单个占用槽位的比特开销可分解为: (1) storage 中 $\log U - \log N$ 比特 payload; (2) $B[i]$ 中变长 MetaEntry, 其中, δ 采用 varint 编码, 长度会随 $|g_I(x) - i|$ 增大而增加, mid_bits 占 $\Theta(\log(K/|I|))$ 比特, router_bits 与 insert_bits 为常数宽度。Facility 编号 r 与 Cubby 容量 $|I|$ 作为查询路径上的上下文信息, 不重复写入每个槽位。整体上来看, 商压缩将 $\pi(x)$ 中与寻址重叠的低位从槽位载荷中抽离, 使附加空间与 $O \log^{(k)} n$ 级预算相容; MiniArray B 的变长编码则使元数据按条目实际所需位数记录, 避免为所有槽位预留固定最大键长^[12,15]。

3.6 插入、查询与删除

插入。计算 $g^*(x) = \pi(x)[1, \log N]$ 与 Facility 编号 r ; 在对应 Facility 的 tail Cubby 内执行 InsertKick (算法 13), 写入 storage、更新 M 与 $B[i]$, 并在 $A[g_I(x)]$ 中 $\text{insert}(\pi(x), j)$ 。tail 满则挂入 1-tiered cubby 并新建 tail, 必要时按算法 8 触发合并^[4,19]。

查询。计算 $g^*(x)$ 定位 Facility 与 Cubby; 令 $i = g_I(x)$, 在 $A[i]$ 中 $\text{query}(\pi(x))$ 得 j ; 访问 $h_j(x)$, 用 payload、 B 、 r 重建 $g^*(x)$ 并与 $\pi(x)$ 比较。该路径仅含常数次 MiniArray 与 Local Query Router 访问, 查询时间为 $O(1)$ ^[4]。

删除。定位元素所在 Cubby 与槽位, 清空 storage, 更新 $A[g_I(x)].\text{delete}(\pi(x))$ 、 M 与 B 。若删除发生在非 tail Cubby, 按算法 9 从 tail 随机抽取元素回填; 回填

写入同样走 k-kick 与 Local Query Router 更新路径，以恢复各 bin 的饱和与深度不变量。

表级扩缩容时， N 、 K 、 r_j 与商压缩上下文变化，须先恢复原始键再按新参数重插，而不能直接复制旧槽位的 payload 与 $B^{[2,4]}$ 。

3.7 本章小结

本章给出分层哈希表的主要结构与算法: Facility 级 MiniArray 以 B 树、Rank/Select 与 Split/Merge 维护变长元数据^[24,26]；多层级 Cubby 与 tail 机制定义 tier 调度与删除回填不变量^[6-7]；k-kick 经偏好序列、饱和判定、InsertKick 与 ReInsert 实现 $O(k)$ 交换插入^[4,9]；Local Query Router 经 $A[i]$ 前缀树将查询降为 $O(1)$ ^[8,29]。四组件同步更新规则贯穿插入、踢出、删除回填与扩缩容重插；第四章报告各参数下的性能测量。

第四章 系统性能实验

4.1 实验目的与环境

本章基于第二、三章给出的结构与参数设定，对已实现的哈希系统进行性能测试和功能验证。实验围绕操作延迟、k-kick 踢出次数、多层级 Cubby 向下拆分（rebuild_down）与向上合并（rebuild_up）三类指标展开，用于检查第三章各模块在实际运行中的效果^[2,4]。

实验的系统代码基于 C++23 标准开发实现，在 AMD Ryzen 7 6800H 处理器、Windows 10 环境下运行。元素规模 $n \in \{10^3, 10^4, 10^5\}$ ，Facility 规模 K 按分组取多档（基准 $K = \log^3 n$ ），k-kick 最大深度 $k \in \{3, 4, 5\}$ （基准 $k = 4$ ），NODE_MAX_BITS 常数 $c = 2$ 。键由 std::mt19937_64 在固定种子 seed=1（经 seed1/seed2/seed3 派生表内哈希参数）下均匀生成；实验报告单次 workload 内各操作类型的平均延迟。

4.2 实验方案

4.2.1 实验分组与工作量

实验分为四组，如表 4-1 所示。规模 n 组与 K 组、 k 组采用键均匀分布于各 Facility 的混合增删负载；Cubby 动态重构组按纯插入、纯删除及不同插入/删除比例构造 workload，分别触发 rebuild_up 与 rebuild_down。

表 4-1 实验分组与主要验证内容

分组	负载方式	主要验证内容
规模 n	均匀分布、混合增删	操作延迟、踢出、rebuild
Facility K	均匀分布、混合增删	操作延迟、踢出
k-kick	均匀分布、高负载	操作延迟、踢出
Cubby	Insert/Delete/混合比例	rebuild_down、rebuild_up、重构耗时

4.2.2 评价指标

为量化评估系统的基础性能，实验记录并分析以下指标：

- 操作延迟：插入、命中查询和删除的平均延迟。该指标用于衡量哈希系统基础操作的运行性能；
- k-kick: workload 内累计踢出次数，以及单次插入/删除搬移最大次数 insert_moved_max、delete_moved_max。该指标用于观察 k-kick 核心模块的具体表现，并判断模块的基础踢出功能是否生效；
- j-tiered Cubby: rebuild_down（向下拆分）与 rebuild_up（向上合并）触发次数。该指标用于验证多层级的 Cubby 的合并与拆分功能是否正常触发；

4.3 规模 n 实验

固定 $K = \log^3 n$ 、 $k = 4$ 、 $c = 2$ ，令 n 从 10^3 增至 10^5 。表 4-2 给出各档测量结果。

表 4-2 规模 n 实验结果（ $K = \log^3 n$ ， $k = 4$ ，键均匀分布）

n	ins (μ s)	qry (μ s)	del (μ s)	总踢出	rebuild_down	rebuild_up
10^3	49.1	2.2	66.0	80	11	179
10^4	42.0	2.7	71.6	1157	77	930
10^5	60.8	5.1	2051.9	10485	11447	21502

表中 rebuild_down 对应向下拆分的次数，rebuild_up 对应向上合并的次数。

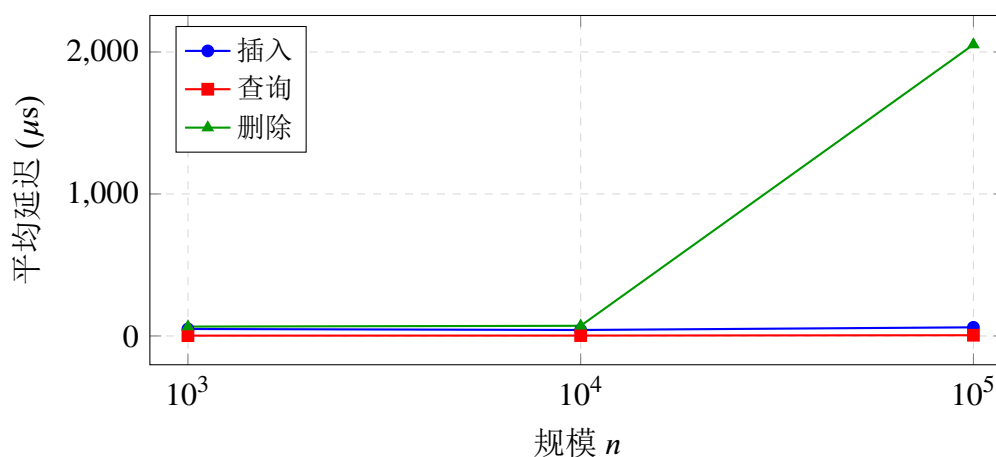


图 4-1 规模 n 实验中三种操作平均延迟（ $K = \log^3 n$ ， $k = 4$ ，键均匀分布）

随 n 增大，命中查询延迟由 $2.2\ \mu\text{s}$ 升至 $5.1\ \mu\text{s}$ ，相对三个数量级跨度增幅平缓，与 Local Query Router 的常数步定位一致^[4,29]。插入延迟在 $42.0\text{--}60.8\ \mu\text{s}$ 之间， $n = 10^3$ 略高（ $49.1\ \mu\text{s}$ ），与 Facility 数较少、单桶局部竞争有关。删除延迟由 $66.0\ \mu\text{s}$ 增至 $2051.9\ \mu\text{s}$ ，增幅远高于插入： $n = 10^5$ 时 rebuild 合计约 3.3 万次，删除路径上的 tail 回填、tier 拆分及 MiniArray 同步均叠加在热路径上（见图 4-1）。累计踢出由 80 增至 1.0×10^4 ，rebuild_down 由 11 增至 11447 次、rebuild_up 由 179 增至 21502 次；大规模下 j-tiered Cubby 维护最为频繁^[6-7]。

4.4 Facility 规模 K 实验

固定 $k = 4$ ，在 $n \in \{10^3, 10^4, 10^5\}$ 三档下各取多组 K 比较三操作延迟与踢出。表 4-3–4-5 汇总实测结果（键均匀分布）。

表 4-3 Facility 规模 K 实验（ $n = 10^3$ ， $k = 4$ ）

K	ins (μs)	qry (μs)	del (μs)	总踢出
16	22.6	2.1	52.8	112
64	49.9	2.0	67.7	60
256	51.9	1.9	66.0	58

表 4-4 Facility 规模 K 实验（ $n = 10^4$ ， $k = 4$ ）

K	ins (μs)	qry (μs)	del (μs)	总踢出
64	56.9	2.8	75.9	813
256	54.1	2.7	70.9	916
1024	52.9	2.6	72.0	963

表 4-5 Facility 规模 K 实验（ $n = 10^5$ ， $k = 4$ ）

K	ins (μs)	qry (μs)	del (μs)	总踢出
256	98.9	4.2	504.9	6889
1024	185.2	3.4	518.3	6883
4096	64.9	3.1	127.1	8756

$n = 10^3$ 时 $K = 16$ 插入最快（ $22.6\ \mu\text{s}$ ），但踢出最多（112 次）； $K \geq 64$ 时踢出降至约 60 次，增删延迟略升，体现小 n 下 Facility 过细反而增加路由与维护开销。 $n = 10^4$ 三档 K 下查询均在 $2.6\text{--}2.8\ \mu\text{s}$ ，踢出随 K 增大变化不大（813–963），说明中等规模下 K 对均匀负载的边际影响有限。 $n = 10^5$ 时 $K = 4096$ 删除延迟

(127.1 μs) 明显低于 $K = 256/1024$ (504–518 μs)，与大规模 rebuild 主要落在较小 K 档、大 Facility 缓解局部冲突一致； $K = 1024$ 插入异常偏高 (185.2 μs) 可能与 Facility 数与 Cubby 布局的组合有关，部署时宜结合目标 n 与负载率选取 $K^{[2,4]}$ 。

4.5 k-kick 深度 k 实验

对 $k \in \{3, 4, 5\}$ 在各规模下执行相同高负载混合增删 workload: $n = 10^3$ 取 $K = 64$, $n = 10^4$ 取 $K = 256$, $n = 10^5$ 取 $K = 4096$ 。表 4-6–4-8 给出完整结果；图 4-2 汇总三档规模下 bits/key 随 k 的变化。

表 4-6 k-kick 深度 k 实验 ($n = 10^3$, $K = 64$, 高负载)

k	ins (μs)	qry (μs)	del (μs)	总踢出
3	71.2	2.1	85.8	61
4	73.0	2.1	88.7	104
5	77.5	2.3	97.7	194

表 4-7 k-kick 深度 k 实验 ($n = 10^4$, $K = 256$, 高负载)

k	ins (μs)	qry (μs)	del (μs)	总踢出
3	54.5	2.7	72.6	265
4	55.3	2.6	72.6	900
5	56.4	2.5	70.2	1587

表 4-8 k-kick 深度 k 实验 ($n = 10^5$, $K = 4096$, 高负载)

k	ins (μs)	qry (μs)	del (μs)	总踢出
3	60.3	3.1	2229.4	2772
4	62.6	3.2	2003.9	8707
5	69.1	3.3	1734.3	14986

insert_moved_max 与 delete_moved_max 各档均不超过 k 。

三档规模下累计踢出均随 k 增大而上升： $n = 10^3$ 由 61 增至 194, $n = 10^4$ 由 265 增至 1587, $n = 10^5$ 由 2772 增至 14986^[4,9]。查询延迟在各规模均不超过 3.3 μs , kick 深度主要作用于增删路径。 $n = 10^5$ 删除延迟达毫秒级, 且 $k = 5$ 时 (1734.3 μs) 低于 $k = 3/4$, 与大规模 rebuild 叠加、较深 kick 链在特定负载下分摊搬移的实测表现一致；中小规模下增删延迟随 k 变化相对平缓^[2,4]。图 4-2, 表

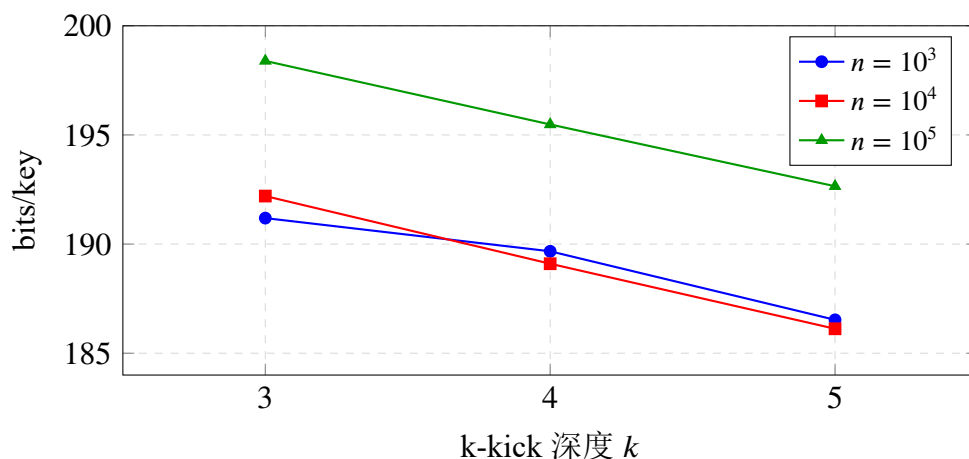


图 4-2 k-kick 深度 k 实验中 bits/key

明，随着 k-kick 树深度的增加，每键占用的内存呈现下降趋势，这与论文核心观点相符。

4.6 多层次 Cubby 动态重构实验

固定 $k = 4$ ，在 $n \in \{10^3, 10^4, 10^5\}$ 下分别取 $K = 64, 256, 4096$ ，按表 4-9-4-11 四种 workload 执行操作序列，统计 rebuild 次数与单次重构平均耗时。

表 4-9 多层次 Cubby 动态重构 ($n = 10^3$, $K = 64$)

Workload	rebuild_down	rebuild_up	平均重构耗时 (ms)
Insert-only	1	31	3.72
Delete-only	28	0	6.24
80/20 混合	0	24	3.71
50/50 混合	6	25	3.82

表 4-10 多层次 Cubby 动态重构 ($n = 10^4$, $K = 256$)

Workload	rebuild_down	rebuild_up	平均重构耗时 (ms)
Insert-only	0	384	3.47
Delete-only	384	2	6.10
80/20 混合	8	355	3.32
50/50 混合	68	377	3.35

三档规模下规律一致：纯插入以 rebuild_up 为主 ($n = 10^3$ 为 31 次， $n = 10^5$ 为 3591 次)，纯删除以 rebuild_down 为主 (28–4211 次)，对应 Cubby 随元素增减而向上合并或向下拆分^[6-7]。80/20 混合负载在各规模均以 rebuild_up 为主；50/50 混合时两方向均显著（如 $n = 10^4$ 为 68/377， $n = 10^5$ 为 375/3504），删除

表 4-11 多层级 Cubby 动态重构 ($n = 10^5$, $K = 4096$)

Workload	rebuild_down	rebuild_up	平均重构耗时 (ms)
Insert-only	0	3591	5.64
Delete-only	4211	621	9.76
80/20 混合	177	3588	5.52
50/50 混合	375	3504	5.13

比例升高使 rebuild_down 占比上升。平均重构耗时由 3.3–6.2 ms ($n \leq 10^4$) 增至 5.1–9.8 ms ($n = 10^5$)，仍远低于大规模混合 workload 中单次删除的均摊延迟。

4.7 本章小结

本章在 n 、Facility 规模 K 、k-kick 深度 k 及 Cubby 动态重构四组设定下，于 $n \in \{10^3, 10^4, 10^5\}$ 完成测量。表 4-2 与图 4-1 显示查询延迟由 2.2 μ s 升至 5.1 μ s， $n = 10^5$ 删除延迟达 2051.9 μ s； K 、 k 与 Cubby 三组实验在表 4-3–4-11 中给出三档规模对照，insert_moved_max 各档均不超过 k ，纯插入/纯删除 workload 分别主导 rebuild_up 与 rebuild_down。上述结果与 Local Query Router 常数步查询设计及 k-kick 交换上界在实测中可对照。

第五章 总结与展望

5.1 工作总结

本文以 Bender 等人提出的最优时间/空间权衡框架^[4]为理论基础，完成了分层哈希表的结构与算法设计，并据此实现简化系统、开展性能实验。主要难点在于将 MiniArray、k-kick 与 Local Query Router 联调为统一原型，并在 Facility-Cubby-Slot 分层组织下保持辅助结构同步不变量。

在设计层面，第二、三章给出了 Facility-Cubby-Slot 分层组织、MiniArray、Cubby 内 k-kick、Local Query Router 和商压缩等模块的实现方案。MiniArray 以 B 树与 Rank/Select 维护变长元数据；k-kick 在逻辑 bin 间通过有限次踢出完成插入；Local Query Router 以前缀树将查询映射到目标槽位；storage、MiniArray B 、 M 与 Router 在每次搬移后同步更新。

实验测量与配置见第四章表 4-2–4-11 及图 4-1；kick 深度变化时踢出链的最大次数仍不超过对应的 k ，与第二章交换上界一致。

综上，本文完成了从理论结构到可运行原型的设计与实现，记录了工程简化相对严格模型的差异，并给出可复现的测量数据与实验参数。

5.2 不足与局限

受时间与实验条件限制，本文仍存在以下不足：

- 未与 IcebergHT^[7]、PaCHash^[15] 等同类系统进行同机对比，尚难从相对角度评价本原型的工程竞争力；
- 插入与删除延迟高于查询（ $n = 10^5$ 时分别为 $60.8 \mu s$ 、 $2051.9 \mu s$ ，查询 $5.1 \mu s$ ），与 MiniArray 维护、k-kick 踢出及 tier 重构等热路径相关；
- 第四章合成的 workload 存在分布理想化、阶段化访问模式、规模与场景覆盖窄、缺少外部基线与统计稳健性等局限；

- 实验 workload 对键偏斜、大规模数据及 active/old 双表渐进迁移等场景覆盖不足，结论外推范围受测量方式的限制。

5.3 展望

本文在 Bender 等人的紧凑哈希框架下完成简化实现与四组测量，归纳工程简化相对理论模型的差异；后续工作可从以下几方面展开：在更大规模与偏斜负载下观察 tier 重建与双表迁移对尾延迟的影响；对插入、删除热路径做针对性优化；选取一至两个紧凑哈希开源实现进行同机对比；补充进程级内存与吞吐的端到端测量，更全面地评估空间-时间权衡^[11,15]。

参考文献

- [1] KNUTH D E. The Art of Computer Programming, Volume 3: Sorting and Searching[M]. Reading, MA: Addison-Wesley, 1973.
- [2] BENDER M A, FARACH-COLTON M, KUSZMAUL J, et al. Modern Hashing Made Simple[C/OL]//Proceedings of the 2024 SIAM Symposium on Simplicity in Algorithms (SOSA 2024). SIAM, 2024: 363-373. DOI: 10.1137/1.9781611977936.33.
- [3] 王楠, 吴云. 面向持久化键值数据库的自适应热点感知哈希索引[J/OL]. 计算机应用研究, 2024, 41(1): 226-230. DOI: 10.19734/j.issn.1001-3695.2023.04.0188.
- [4] BENDER M A, FARACH-COLTON M, KUSZMAUL J, et al. On the Optimal Time/Space Tradeoff for Hash Tables[C/OL]//Proceedings of the 54th Annual ACM SIGACT Symposium on Theory of Computing. Rome, Italy: ACM, 2022: 1284-1297. DOI: 10.1145/3519935.3519969.
- [5] KUSZMAUL W, WALZER S. Space Lower Bounds for Dynamic Filters and Value-Dynamic Retrieval[C/OL]//Proceedings of the 56th Annual ACM Symposium on Theory of Computing (STOC 2024). ACM, 2024: 1153-1164. DOI: 10.1145/3618260.3649649.
- [6] BENDER M A, CONWAY A, FARACH-COLTON M, et al. Iceberg Hashing: Optimizing Many Hash-Table Criteria at Once[J/OL]. Journal of the ACM, 2023, 70(6): 40:1-40:51. DOI: 10.1145/3625817.
- [7] PANDEY P, BENDER M A, CONWAY A, et al. IcebergHT: High Performance Hash Tables Through Stability and Low Associativity[C/OL]//Proceedings of the ACM on Management of Data: Proceedings of the ACM SIGMOD International Conference on Management of Data: vol. 1: 1. Seattle, WA, USA: ACM,

- 2023:47:1-47:26. DOI: 10.1145/3588727.
- [8] DIETZFELBINGER M, PAGH R. Succinct Data Structures for Retrieval and Approximate Membership[C/OL]//Lecture Notes in Computer Science: Proceedings of the 35th International Colloquium on Automata, Languages, and Programming (ICALP 2008): vol. 5125. Springer, 2008: 385-396. DOI: 10.1007/11523468_30.
 - [9] PAGH R, RODLER F F. Cuckoo Hashing[J/OL]. Journal of Algorithms, 2004, 51(2): 122-144. DOI: 10.1016/j.jalgor.2004.03.002.
 - [10] LEHMANN H P, SANDERS P, WALZER S. SicHash – Small Irregular Cuckoo Tables for Perfect Hashing[C/OL]//Proceedings of the Symposium on Algorithm Engineering and Experiments (ALENEX 2023). SIAM, 2023: 176-189. DOI: 10.1137/1.9781611977561.ch15.
 - [11] LEHMANN H P, MÜLLER T, PAGH R, et al. Modern Minimal Perfect Hashing: A Survey[J/OL]. ACM Computing Surveys, 2026, 58(10): 251:1-251:36. DOI: 10.1145/3797036.
 - [12] LEHMANN H P, SANDERS P, WALZER S. ShockHash: Towards Optimal-Space Minimal Perfect Hashing Beyond Brute-Force[C/OL]//Proceedings of the Symposium on Algorithm Engineering and Experiments (ALENEX 2024). SIAM, 2024: 194-206. DOI: 10.1137/1.9781611977929.15.
 - [13] HERMANN S, LEHMANN H P, PIBIRI G E, et al. PHOBIC: Perfect Hashing with Optimized Bucket Sizes and Interleaved Coding[C/OL]//LIPIcs: Proceedings of the 32nd Annual European Symposium on Algorithms (ESA 2024): vol. 308. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2024: 69:1-69:17. DOI: 10.4230/LIPIcs.ESA.2024.69.
 - [14] LEHMANN H P, SANDERS P, WALZER S, et al. Combined Search and Encoding for Seeds, with an Application to Minimal Perfect Hashing[C/OL]//LIPIcs: Proceedings of the 33rd Annual European Symposium on Algorithms (ESA 2025): vol. 351. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2025: 109:1-109:18. DOI: 10.4230/LIPIcs.ESA.2025.109.

- [15] KURPICZ F, LEHMANN H P, SANDERS P. PaCHash: Packed and Compressed Hash Tables[C/OL]//Proceedings of the Symposium on Algorithm Engineering and Experiments (ALENEX 2023). SIAM, 2023: 162-175. DOI: 10.1137/1.9781611977561.ch14.
- [16] 熊铁翔, 蒋筱斌, 张珩, 等. 支持分页显存的高性能哈希表索引系统[J/OL]. 计算机系统应用, 2022, 31(9): 82-90. DOI: 10.15888/j.cnki.csa.008664.
- [17] AAMAND A, KNUDSEN J B T, THORUP M. Load Balancing with Dynamic Set of Balls and Bins[C/OL]//Proceedings of the 53rd Annual ACM SIGACT Symposium on Theory of Computing (STOC 2021). ACM, 2021: 1262-1275. DOI: 10.1145/3406325.3451107.
- [18] BANSAL N, KUSZMAUL W. Balanced Allocations: The Heavily Loaded Case with Deletions[C/OL]//Proceedings of the 63rd IEEE Annual Symposium on Foundations of Computer Science (FOCS 2022). IEEE, 2022: 801-812. DOI: 10.1109/FOCS54457.2022.00081.
- [19] MITZENMACHER M, RICHA A, SITARAMAN R. The Power of Two Random Choices: A Survey of Techniques and Results[G]//Handbook of Randomized Computing. Springer, 2001: 255-312.
- [20] PAGH R, PAGH A, RUZIC M. Linear Probing with Constant Independence [J/OL]. SIAM Journal on Computing, 2009, 39(3): 1107-1120. DOI: 10.1137/070702278.
- [21] LUBY M, RACKOFF C. How to Construct Pseudorandom Permutations from Pseudorandom Functions[J/OL]. SIAM Journal on Computing, 1988, 17(2): 373-386. DOI: 10.1137/0217022.
- [22] CARTER L, WEGMAN M N. Universal Classes of Hash Functions[J/OL]. Journal of Computer and System Sciences, 1979, 18(2): 143-154. DOI: 10.1016/0022-0000(79)90044-8.
- [23] THORUP M, ZHANG Y. Tabulation-Based 5-Independent Hashing with Applications to Linear Probing and Second Moment Estimation[J/OL]. SIAM Journal

- on Computing, 2012, 41(2): 293-331. DOI: 10.1137/100800774.
- [24] BAYER R, MCCREIGHT E M. Organization and Maintenance of Large Ordered Indexes[J/OL]. *Acta Informatica*, 1972, 1(1): 173-189. DOI: 10.1007/BF00288683.
 - [25] PAGH R. Hashing with Limited Independence[C]// *Proceedings of the 12th Annual ACM-SIAM Symposium on Discrete Algorithms (SODA 2001)*. SIAM, 2001: 830-839.
 - [26] JACOBSON G. Space-efficient Static Trees and Graphs[C/OL]// *Proceedings of the 30th Annual Symposium on Foundations of Computer Science (FOCS 1989)*. IEEE, 1989: 549-554. DOI: 10.1109/SFCS.1989.63533.
 - [27] CONWAY A, FARACH-COLTON M, KUSZMAUL W. A Unified Approach to Dynamic Optimality for Linear Probing and Robin Hood Hashing[C/OL]// *LIPIcs: Proceedings of the 26th Annual European Symposium on Algorithms (ESA 2018)*: vol. 112. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2018: 29:1-29:14. DOI: 10.4230/LIPIcs.ESA.2018.29.
 - [28] 王天宇, 徐云, 王彪. 基于位图的键值存储哈希优化[J/OL]. *计算机应用研究*, 2023, 40(7): 2106-2110. <http://www.c-s-a.org.cn/1003-3254/8664.html>.
 - [29] MORRISON D R. PATRICIA—Practical Algorithm To Retrieve Information Coded in Alphanumeric[J/OL]. *Journal of the ACM*, 1968, 15(4): 514-534. DOI: 10.1145/321066.321078.

致 谢

行文至此，落笔为终。感谢我的导师、感谢身边的每一个同学、感谢遇到的每一个人。凡是过往，皆为序章，很喜欢看到的一句话，人生如棋，落子无悔。愿我们跃入人海，各自灿烂。